

Constrained RL for Safe Autonomous Navigation with COLREG Rules

AI&Robotics and XAI Project

Enrico Loda, Luca Panariello

Department of Computer Science
Master's Degree in Artificial Intelligence
University of Verona

Academic Year 2025/2026

1. Introduction

1.1 Project Overview

2. Environment & Implementation

2.1 Environment & Task Setup

2.2 PPO Method

2.3 Generalized Advantage Estimation

2.4 Loss Computation

2.5 CAGrad

2.6 Agent Architecture

2.7 Training & Evaluation Setup

3. XAI & Rule Monitoring

3.1 Formal Rule Monitoring via STL

3.2 Baselines & Comparison Setup

4. Results and Discussion

4.1 Training Curves

4.2 Evaluation results

4.3 Test results

5. Conclusion

Project Overview

- **Goal:** Training an autonomous boat in **Unity** to navigate safely while following maritime traffic rules (COLREGs: safe distance, safe speed, stand-on).
- **Method:** Constrained **PPO** with separate networks for task reward and safety costs, using CAGrad to resolve conflicting objectives.
- **Explainability:** Using **Signal Temporal Logic (STL)** to provide clear, numerical safety scores (ρ) for each rule.
- **Baselines:** Evaluated across multiple seeds against a standard **Reward Shaping** approach and **Random Constraint (CMORL)** selection.

Section 2

Environment & Implementation

Environment & Task Setup 1/8

- **Environment:** 3D simulated environment built in Unity with the ML-Agents toolkit.
- **Observation space ($I = 37$):**
 - 20 physical states (velocities, orientation, target position);
 - 14 raycast inputs (7 sensor rays $\times$ [distance, hit flag]);
 - 3 Markovian STL flags (tracking active rule monitoring for R1, R2, R6).
- **Action space ($A = 2$):** continuous control in $[-1, 1]$ for throttle and steering, driving differential thrusters.
- **Encounter scenarios:** dynamic obstacle avoidance requiring strict compliance with COLREG rules (R1 - safe distance, R2 - safe speed, and R6 - stand-on).

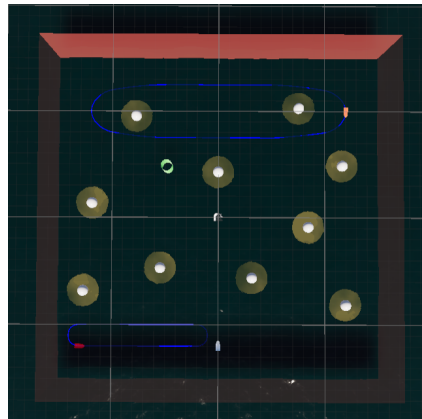
Environment & Task Setup 2/8

- **Arena Layout:**

- 15 × 15 m square water arena.
- Realistic water surface rendered via Unity's High Definition Render Pipeline (HDRP).

- **Navigation Goal:**

- Represented by a green cylinder.
- Position is randomized at the start of every episode to encourage path versatility.



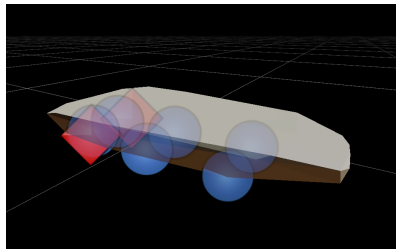
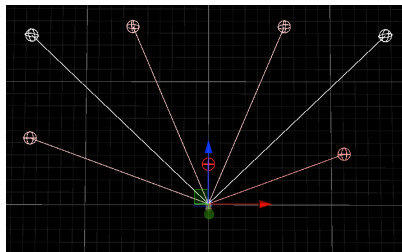
Environment & Task Setup 3/8

- **Static Obstacles (Spheres):**
 - Distributed within the arena in precise locations.
 - Their exact position is randomized at every epoch along their yellow circles.
- **Dynamic Intruders (Obstacle Boats):**
 - Two active vessels following custom trajectories generated via splines.
 - Designed to consistently induce multi-vessel encounters and collision risks.
- **Episodic Domain Randomization:**
 - Speed and physical scale of the obstacle boats trajectories vary at each episode.
 - Prevents the agent from memorizing fixed collision patterns, enforcing reactive rule compliance (COLREGs).

Environment & Task Setup 4/8

The vessel integrates three core physical and sensory systems:

- **7 raycast sensors** distributed along the bow and sides, providing **real-time distance** readings and binary **obstacle detection** flags.
- **Two independent thrusters** configured as a **differential drive**, providing direct continuous control over **linear acceleration** (throttle) and **angular steering**.
- **6 floating points** placed around the hull, where a dedicated physics script computes dynamic **buoyancy forces** and ensures realistic **pitch and roll stability** during wave interactions.



Environment & Task Setup 5/8

Vertical Stability F_v is computed at each of the 6 floaters f_i :

$$F_v(f_i) = \underbrace{b \cdot \frac{d_i}{d_{\max}} \cdot \frac{mg}{N_{\text{floaters}}}}_{\text{Buoyancy force}} - \underbrace{c \cdot v_{i,y} \cdot \frac{d_i}{d_{\max}}}_{\text{Vertical damping}}$$

- **Buoyancy (Push Up):** Restores equilibrium by pushing upward proportionally to floater depth d_i to keep the hull afloat. Where d_i is the depth of the floater f_i , m is the mass, g is the gravity, and b is a scaling factor.
- **Damping (Shock Absorber):** Opposes vertical speed $v_{i,y}$ of the floater f_i to dissipate wave energy and prevent unnatural bouncing. c is a fixed scaling factor.

Environment & Task Setup 6/8

- **Lateral Resistance (Keel Effect):**

- In standard rigid-body physics, floating hulls drift sideways during turns like a car on ice (the “soap-bar effect”).
- An opposing force is applied along the local x -axis, leaving forward thrust (z -axis) unaffected:

$$F_x = -c_{\text{side}} \cdot v_{\text{boat},x} \quad (c_{\text{side}} = 5.0)$$

- **Differential Propulsion:**

- Two continuous actions, **throttle** and **steering**, are mapped to the individual thrusters:

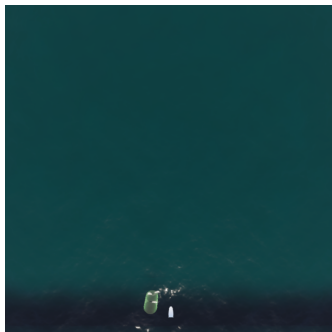
$$T_{\text{left}} = \text{throttle} + \text{steering}$$

$$T_{\text{right}} = \text{throttle} - \text{steering}$$

- **Asymmetric Reverse:** When $\text{throttle} < 0$, its value is reduced by a factor of 0.3 to allow slower reverse maneuvers.
- Commands are clamped to $[-1, 1]$ and scaled by the maximum thrust force ($F_{\text{max}} = 25 \text{ N}$).

Environment & Task Setup 7/8

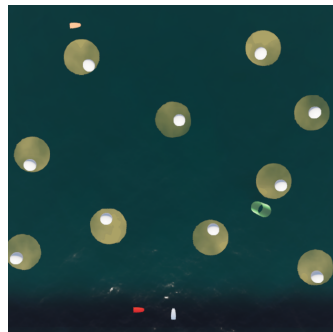
- Curriculum Learning Strategy:



Stage 0



Stage 1 ($\approx 252k$)



Stage 2 ($\approx 502k$)

Environment & Task Setup 8/8

- **Unified Reward Formulation** $\mathcal{R} = \sum_{t=0}^{T_{\max}} r_t$ such that:

$$r_t = \begin{cases} +R_{\max} & \text{if target reached (success)} \\ -R_{\max} & \text{on collision (obstacle, boat, or wall)} \\ \frac{d_{t-1} - d_t}{d_{\text{spawn}}} - \frac{R_{\max}}{T_{\max}} & \text{step reward} \end{cases}$$

where $R_{\max} = 10$ and maximum episode duration $T_{\max} = 5000$ steps (50 s).

- **Initial distance normalization:** Scaling by d_{spawn} caps total progress reward at +1.0, ensuring identical reward scale whether the target spawns near or far.
- **Timestep Penalty** $\frac{R_{\max}}{T_{\max}}$ is used to penalizes idling and wide loops, incentivizing the agent to find the shortest viable trajectory.

PPO Method 1/2

- **Proximal Policy Optimization (PPO):**

- An on-policy actor-critic method bounding policy update steps via a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

- Prevents catastrophic performance collapse while retaining first-order computational efficiency.
- Maximizes sample efficiency through multi-epoch mini-batch updates over buffered rollout data without violating trust-region boundaries.

- **Variable Mapping:**

- $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$: Probability ratio between new and old policy distributions.
- $\hat{A}_t$: Action advantage estimate computed via GAE.
- $\epsilon = 0.2$: Clipping hyperparameter restricting policy changes.

PPO Method 2/2

- **From Standard PPO to Decoupled Multi-Critic:**

- Standard PPO relies on a single scalar critic, which fails when conflating navigation progress with distinct maritime safety rules.
- To decouple conflicting objectives, the architecture employs **one Policy** for unified hull control, **one Value network** for uncorrupted target progression, and **three Cost networks** to independently capture the distinct physical regimes and temporal horizons of rules R_1 , R_2 , and R_6 .
- Decoupling produces isolated per-rule advantage signals, providing clean, independent gradient vectors for conflict resolution via CAGrad.

Generalized Advantage Estimation 1/2

- **Temporal-Difference (TD) Residual (δ_t):**

$$\delta_t = r_t + \gamma V(s_{t+1})m_t - V(s_t)$$

- Measures the immediate prediction error (“surprise”) at step t , where s_t is the state, $m_t \in \{0, 1\}$ is the end episode flag, r_t is the immediate feedback signal (task reward or safety cost) and $V(s_t)$ is the estimated expected cumulative return from s_t .
- **Recursive Generalized Advantage Estimation (GAE) Formulation ($\hat{A}_t$):**

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} = \delta_t + \gamma \lambda m_t \hat{A}_{t+1}^{\text{GAE}}$$

- Computes an exponentially decayed sum of future residuals backward across the rollout.
- $\gamma = 0.995$: Temporal discount factor for future horizons.
- $\lambda = 0.95$: Trace decay parameter balancing bias and variance.

Generalized Advantage Estimation 2/2

Critic targets for MSE loss ($L = \|\hat{Y}_t - V(s_t)\|^2$):

$$\hat{Y}_t = \hat{A}_t + V(s_t)$$

Task Critic (V^ϕ):

- Signal: environment reward r_t
- Advantage: $A^{\text{reward}} = \text{GAE}(r, V^\phi)$
- Objective: navigation target reaching

Safety Critics ($V_{R_k}^{\psi_k}$):

- Signal: STL violation penalty c_{R_k}
- Advantage: $A_{R_k}^{\text{cost}} = \text{GAE}(c_{R_k}, V_{R_k}^{\psi_k})$
- Monitored rules: R_1, R_2, R_6

Loss Computation

- **Robustness-Driven Loss Switching:**

$$L(\theta) = \begin{cases} L^{\text{CLIP}}(\theta, A^{\text{reward}}) - c_{\mathcal{H}}\mathcal{H}(\pi_{\theta}) & \text{if } \rho_k \geq 0 \quad (\text{Nominal Mode}) \\ L^{\text{CLIP}}(\theta, -A_{R_k}^{\text{cost}}) - c_{\mathcal{H}}\mathcal{H}(\pi_{\theta}) & \text{if single } \rho_k < 0 \quad (\text{Single Violation}) \\ \text{CAGrad}(\{L^{\text{CLIP}}(\theta, -A_{R_i}^{\text{cost}})\}_{i \in \mathcal{V}}) & \text{if multiple } \rho_k < 0 \quad (\text{Multiple Violations}) \end{cases}$$

CAGrad

- **Gradient Conflict Problem:**

- Competing rules often produce opposing action gradients (e.g., accelerating to clear safe distance vs. braking for safe speed).
- Standard summation ($\sum g_i$) causes gradient cancellation, leading to actuator paralysis.

- **CAGrad Formulation ($c = 0.5$):**

$$g_m = \text{CAGrad}(\{g_{R_i}\}_{i \in \mathcal{V}}) = \arg \max_g \min_{k \in \mathcal{V}} \langle g, g_{R_k} \rangle \quad \text{subject to} \quad \|g - g_{\text{avg}}\| \leq c \|g_{\text{avg}}\|$$

- $g_{R_k} = \nabla_{\theta} L^{\text{CLIP}}(\theta, -A_{R_k}^{\text{cost}})$: Gradient vector for rule R_k .
- $g_{\text{avg}} = \frac{1}{|\mathcal{V}|} \sum_{k \in \mathcal{V}} g_{R_k}$: Unweighted mean gradient direction.
- $c \in [0, 1]$: Conflict-aversion parameter bounding deviation from the average.

- **Key Properties:**

- Maximizes the worst-case progress across all active constraints.
- Guarantees that fulfilling one safety rule does not compromise another.

Agent Architecture

- **Multi-critic structure:**
 - **1 Policy Network (π_θ):** Continuous actions (throttle and steering) with learnable standard deviation.
 - **1 Value Network (V^ϕ):** Predicts environmental return for task completion.
 - **3 Cost Networks ($V_{R1}^{\psi_1}, V_{R2}^{\psi_2}, V_{R6}^{\psi_3}$):** Predict expected cumulative penalties separately for Safe Distance, Safe Speed, and Stand-On.
- **Common backbone:** 2-layer MLP with 128 hidden units per layer, ReLU activations, and orthogonal weight initialization.
- **Output heads:**
 - **Policy:** outputs mean action μ with a learnable log standard deviation parameter σ ;
 - **Value & Costs:** single linear scalar heads predicting GAE returns and cumulative discounted costs.

Training & Evaluation Setup 1/5

- **Algorithm:** Constrained PPO with Generalized Advantage Estimation (GAE).
- **Hyperparameter Grid Search:** Conducted on the simplest curriculum stage, with search ranges based on Stable-Baselines3 defaults:
 - **Learning rate:** $\{1 \times 10^{-4}, 3 \times 10^{-4}\}$.
 - **Entropy coefficient:** $\{0, 0.001, 0.005\}$.
 - **Batch size:** $\{64, 128, 256\}$.
 - **Discount factor:** $\gamma = 0.995$, fixed to provide an effective planning horizon of ≈ 200 steps ($1/(1 - \gamma)$).

Training & Evaluation Setup 2/5

- **Best Configurations:**

- Adam Optimizer with Learning Rate $lr = 3 \times 10^{-4}$
- **Entropy** $\mathcal{H} = 0.001$
- **Discount Factor** $\gamma = 0.995$
- **Rollout Size** $N = 2048$
- **Minibatch Size** $B = 256$
- **Cost scale** $\alpha_{cost} = 0.1$
- **Training steps** $S = 2.048M$
- **Safety:** Start to optimize for constrain violations only after $\approx 1M$ steps.

Training & Evaluation Setup 3/5

- **Evaluation protocol:**

- 5 training runs across random seeds (1, 3, 7, 34, 42);
- Evaluate on a dedicated environment (10 episodes per fixed seed) every 5 updates.

- **Saved Checkpoints:**

- **Pre-safety:** Last model saved before the safety activation threshold.
- **Best:** Model achieving the highest evaluation return after safety activation.
- **Last:** Final model obtained at the end of training.

Training & Evaluation Setup 4/5

- **Python–Unity Interaction Loop:**

- Executes synchronous lock-step communication via the ML-Agents API:
 1. **Observation:** Unity evaluates rigid-body hydrodynamics and sensors, after 5-unity step call the Python script by sending state s_t .
 2. **Inference:** Python evaluates policy $\pi_\theta(a_t|s_t)$ and dispatches continuous control actions a_t (thrust and steering) to Unity via `set_actions()`.
 3. **Environment Step:** `env.step()` advances the physical simulation, returning the subsequent state s_{t+1} and immediate reward r_t .
- Transitions accumulate in a rollout buffer ($N = 2048$ steps) before pausing simulation steps to execute network updates.

Training & Evaluation Setup 5/5

- **Episode-Aligned Rollout Buffering:**

- Gathers transitions until reaching buffer capacity ($N \geq 2048$ steps) **and** completing the current episode.
- Prevents trajectory truncation, ensuring complete temporal traces required for accurate offline STL evaluation.

- **STL Robustness to Cost Mapping (RTAMT):**

- Evaluates dense robustness scores ρ_k for rules R_1, R_2, R_6 at episode boundaries.
- Transforms continuous robustness into bounded step penalties:

$$c_k = \tanh(-\rho_k) \cdot \alpha_{\text{cost}}$$

Section 3

XAI & Rule Monitoring

Formal Rule Monitoring via STL 1/4

- **Temporal Logic Monitoring (RTAMT):**
 - Formulates COLREG rules as bounded Signal Temporal Logic (STL) specifications evaluated in dense-time semantics.
 - Discretization: $\Delta t = 0.05 \text{ s}$ (20 Hz decision step), sliding evaluation horizon $H = 80$ steps (4.0 s).
- **Robustness Score (ρ):** Evaluated via RTAMT to quantify safety:
 - $\rho \geq 0$: the rule is fully satisfied (positive safety margin);
 - $\rho < 0$: the rule is violated, with magnitude measuring danger severity.
- **On-Policy Rollout Advantage:**
 - Collecting complete rollout trajectories prior to updates enables exact, retrospective STL evaluation across the full temporal window without requiring future-state estimations.

Formal Rule Monitoring via STL 2/4

- **Monitored COLREG Rules:**

- **R1 (Safe Distance):** Keep a safe separation so that no collision can occur within a predicted time horizon.

$$\varphi_{R_1} = \mathcal{G}_{[0,80]}(s_{R_1} \geq 0.0)$$

- **R2 (Safe Speed):** Regulate speed according to proximity, ensuring enough time for proper maneuvering.

$$\varphi_{R_2} = \mathcal{G}_{[0,80]}(v_{\text{ego}} \leq v_{\text{safe_limit}}) \quad (v_{\text{safe_limit}} = 2.1 \text{ m/s})$$

- **R6 (Stand-On Vessel):** When holding right-of-way (target on left side or being overtaken) with collision risk, maintain course until the situation is resolved.

$$\mathcal{G}_{[0,80]} \left((s_{\text{keep}} \geq 0.0) \implies ((s_{\text{no_turn}} \geq 0.0) \mathcal{U} (s_{\text{keep}} < 0.0)) \right)$$

Formal Rule Monitoring via STL 3/4

- **Safety Signal (s_{R_1}):**

$$s_{R_1} = \min(d_{\min} - d_{\text{safe}}, 1.0 \text{ m}) \quad (d_{\text{safe}} = 2.0 \text{ m})$$

- **Closest Point of Approach (CPA, $t_h = 1.0 \text{ s}$):**

$$d_{\min} = \begin{cases} \|\mathbf{p}_{\text{rel}}\| & t_{\text{cpa}} < 0 \quad (\text{diverging}) \\ \|\mathbf{p}_{\text{rel}} + \mathbf{v}_{\text{rel}} t_h\| & t_{\text{cpa}} > t_h \quad (\text{beyond horizon}) \\ \|\mathbf{p}_{\text{rel}} + \mathbf{v}_{\text{rel}} t_{\text{cpa}}\| & 0 \leq t_{\text{cpa}} \leq t_h \quad (\text{imminent CPA}) \end{cases}$$

Formal Rule Monitoring via STL 4/4

- **Stand-On Activation Conjunction (s_{keep}):**

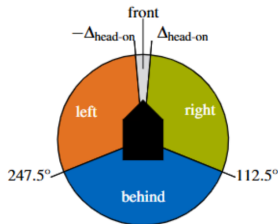
$$s_{\text{keep}} = \min(s_{\text{risk}}, s_{\text{sector}})$$

- **Collision Risk Extraction (s_{risk}):** Evaluates CPA lookahead over an extended horizon ($t_h = 2.0$ s):

$$s_{\text{risk}} = -s_{R_1}(t_h = 2.0 \text{ s})$$

- **Port Sector Detection (s_{sector}):** Relative bearing with port mapped to positive angles: $\theta = \text{atan2}(-p_x, p_z) \in [-180^\circ, 180^\circ]$.

$$s_{\text{sector}} = \frac{1}{k_\theta} \min(\theta - 5.0^\circ, 112.5^\circ - \theta) \quad (k_\theta = 10.0^\circ/\text{m})$$



Baseline Comparison & Trade-offs

- **Compared Approaches:**

- **Our Method (Multi-Critic + CAGrad):** Keeps separate cost networks for R1, R2, and R6, using gradient surgery to balance conflicting rules.
- **Reward Shaping (RS):** Standard PPO where STL penalties are subtracted directly from the reward into a single scalar.

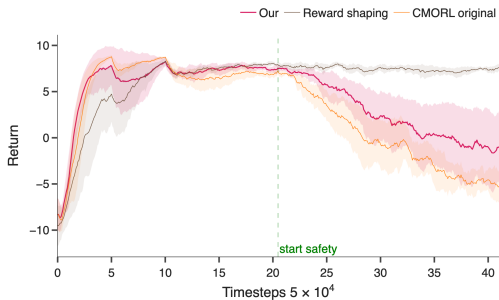
$$r_{RS}^t = r^t - (\text{ReLU}(c_{R_1}^t) + \text{ReLU}(c_{R_2}^t) + \text{ReLU}(c_{R_3}^t))$$

- **Random Constraint (CMORL):** A multi-critic baseline that randomly picks one violated rule per update instead of using CAGrad.

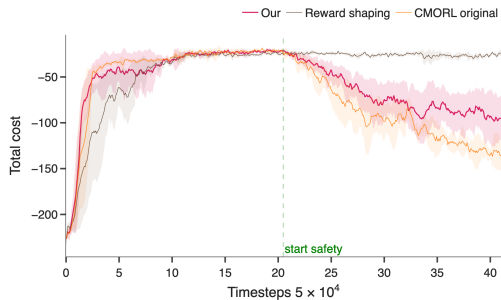
Section 4

Results and Discussion

Training Performance: Return & Costs

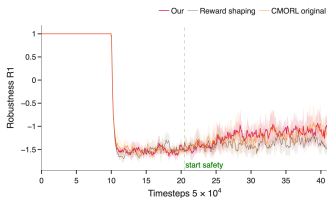


Return

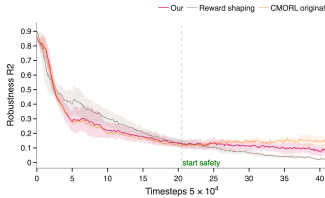


Total Cost

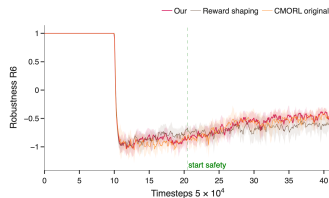
STL Constraints Robustness (ρ)



R1 – Safe Distance

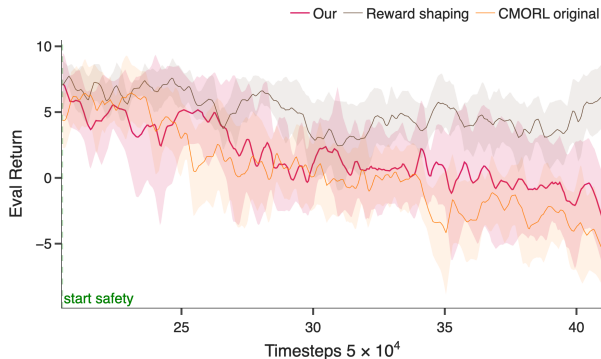


R2 – Safe Speed



R6 – Stand-on

Evaluation Return Curve



- **Our** and **Random CMORL** return tend to degrade after safety activation, due to catastrophic forgetting.
- **Reward shaping** is the only method that improves its eval return after safety.
- The complex STL formulas contribute to this behavior, as every rollout contains at least one constraint violation.

Validation Benchmark: Checkpoint Comparison

Method	Checkpoint	Mean Return	Safe R1 (%)	Safe R2 (%)	Safe R6 (%)	Totally Safe (%)
Our	PRE-SAFETY 1.02M steps	5.32 ± 7.90	50	100	10	10
	BEST 1.29M steps	6.95 ± 5.95	70	100	30	30
	LAST 2.05M steps	-4.91 ± 10.08	70	100	20	20
Reward shaping	PRE-SAFETY 1.02M steps	5.20 ± 7.76	30	100	0	0
	BEST 1.31M steps	7.34 ± 6.00	60	100	30	30
	LAST 2.05M steps	4.44 ± 8.78	50	60	30	20
Random CMORL	PRE-SAFETY 1.02M steps	3.54 ± 8.94	50	80	10	10
	BEST 1.04M steps	7.35 ± 5.98	50	80	20	10
	LAST 2.05M steps	-9.34 ± 3.88	20	100	10	10

Test Benchmark: Generalization Performance

Method	Checkpoint	Mean Return	Safe R1 (%)	Safe R2 (%)	Safe R6 (%)	Totally Safe (%)
Our	PRE-SAFETY 1.02M steps	7.27 ± 6.10	20	90	20	10
	BEST 1.29M steps	9.01 ± 0.74	30	100	30	30
	LAST 2.05M steps	-3.85 ± 10.53	50	100	30	30
Reward shaping	PRE-SAFETY 1.02M steps	1.25 ± 9.59	20	90	20	10
	BEST 1.31M steps	9.27 ± 0.43	30	90	20	10
	LAST 2.05M steps	7.13 ± 5.95	30	60	30	10
Random CMORL	PRE-SAFETY 1.02M steps	9.09 ± 0.91	30	80	30	20
	BEST 1.04M steps	9.30 ± 0.34	30	80	30	20
	LAST 2.05M steps	-11.48 ± 3.02	20	100	20	20

Conclusions

- **Best Safety–Performance Trade-off**

- **Our BEST** checkpoint achieves 30% totally safe episodes in both evaluation and test settings while maintaining acceptable return.
- **Reward Shaping** is the only method that maintains competitive performance at the **LAST** checkpoint.

- **Main Limitations**

- **R1** and **R6** remain significantly harder than safe-speed constraint **R2**.
- **Our** and **Random CMORL** degrade after safety activation due to catastrophic forgetting.

- **Key Takeaway**

- Checkpoint selection is crucial: the final policy is not necessarily the safest or best-performing one.
- Future work should investigate adaptive constraint scheduling and forgetting-mitigation strategies.

Thank you for the attention!

Now let's look at the video